

Assignment #6

Total score: 200

Due date: 10/12/2024

Names: Gabrieliuss (Gabe) Gintalas, Dmitry Bozhko, Reece Bailey

<https://github.com/GabrieliussGintalas/CPSC362-AutomatedStockTrading>

Design Patterns Implementation

1. Adapter Pattern

Location: ``/Backend/data_access/``

- ``models/market_data_adapter.py``: Main adapter class that converts data source interfaces
- ``data_adaptees/yahoo_finance_adaptee.py``: Concrete adaptee for Yahoo Finance
- ``data_source_interface.py``: Interface that defines data access methods

2. Decorator Pattern

Location: ``/Backend/data_access/decorators/``

- ``market_data_decorator.py``: Base decorator class
- ``validation_decorator.py``: Adds data validation
- ``caching_decorator.py``: Adds caching functionality

3. Strategy Pattern

Location: ``/Backend/data_access/models/strategy_pattern/``

- ``tradingAlgos.py``: Contains different trading strategy implementations (SMA, MACD, Bollinger Bands)

4. Publisher-Subscriber Pattern

Location: ``/Backend/data_access/``

- ``price_publisher.py``: Manages price update subscriptions
- ``simulate_market_price.py``: Simulates price changes
- ``price_subscriber.py``: Handles price update subscriptions

SOLID Principles Implementation

1. Single Responsibility Principle (SRP)

Location: ``/Backend/data_access/models/strategy_pattern/tradingAlgos.py``

- Each trading algorithm class has a single responsibility
- Example: SMAStrategy only handles SMA calculations and signals
- TradingAlgo base class defines a single interface for calculating signals

2. Open-Closed Principle (OCP)

Location: ``/Backend/data_access/decorators/market_data_decorator.py``

- MarketDataDecorator allows extending functionality without modifying existing code
- New decorators can be added by inheriting from the base decorator class
- Example: ValidationDecorator and CachingDecorator extend functionality without changing base code

3. Liskov Substitution Principle (LSP)

Location: ``/Backend/data_access/models/strategy_pattern/tradingAlgos.py``

- All strategy classes (SMA, BollingerBands, MACD) can be used interchangeably
- Each maintains the contract defined by TradingAlgo base class
- Example: TradingStrategy can use any algorithm that implements TradingAlgo interface

4. Interface Segregation Principle (ISP)

Location: ``/Backend/data_access/models/market_data_adapter.py``

- MarketDataAdapter implements only the methods it needs
- Each method has a specific purpose without unnecessary dependencies
- Separate interfaces for different functionalities (data fetching, live prices)

5. Dependency Inversion Principle (DIP)

Location: ``/Backend/data_access/models/trading_strategy.py``

- TradingStrategy depends on abstractions (strategy interfaces)
- Strategy selection is done through dependency injection
- Data access layer uses interfaces instead of concrete implementations

(a) The vision statement

A program for individual investors seeking a consistent 10% annual return rate in volatile markets. The Quant Trading System is an automated stock trading system aiming to consistently achieve at least 10% return rates in the FNGU and FNGD markets. Unlike traditional investment strategies, our product will perform trades through an advanced mathematical algorithm that will analyze the market and find patterns to ensure consistent performance in unpredictable market conditions.

(b) all the use cases

ID:	UC-1
Name:	Select Symbol and Date Range
Description:	User selects trading symbol and date range for analysis
Primary Actor:	User
Preconditions:	1) Web application is running and accessible 2) User has a compatible web browser 3) Backend server is operational
PostConditions:	1) Selected symbol is stored in the system 2) Date range is validated and stored 3) Frontend is updated to reflect the selection 4) Backend is notified of the selection
Main Success Scenario: (Normal Flow)	1) User navigates to the trading application webpage 2) System displays two buttons labeled "FNGU" and "FNGD" 3) User clicks on their desired ETF symbol button 4) User enters their desired date range using the date picker 5) System validates the date range is within acceptable bounds 6) System sends the symbol and date selection to the backend

ID:	U-2
Name:	Fetch Market Data
Description:	System automatically fetches and stores market data at market close for each day
Primary Actor:	System

Secondary Actor:	Yahoo Finance API
Preconditions:	1) System has valid API credentials for Yahoo Finance 2) Market is closed for the day (if we want to get the data for today) 3) Internet connection is available 4) System is live and running
PostConditions:	1) Latest market data is stored in JSON format 2) System logs are updated
Main Success Scenario: (Normal Flow)	1) System initiates data fetch at market close 2) System connects to Yahoo Finance API 3) System requests FNGU and/or FNGD data 4) Yahoo Finance API returns requested data 5) System formats data according to JSON schema 6) System saves formatted data to storage 7) System logs successful data fetch operation

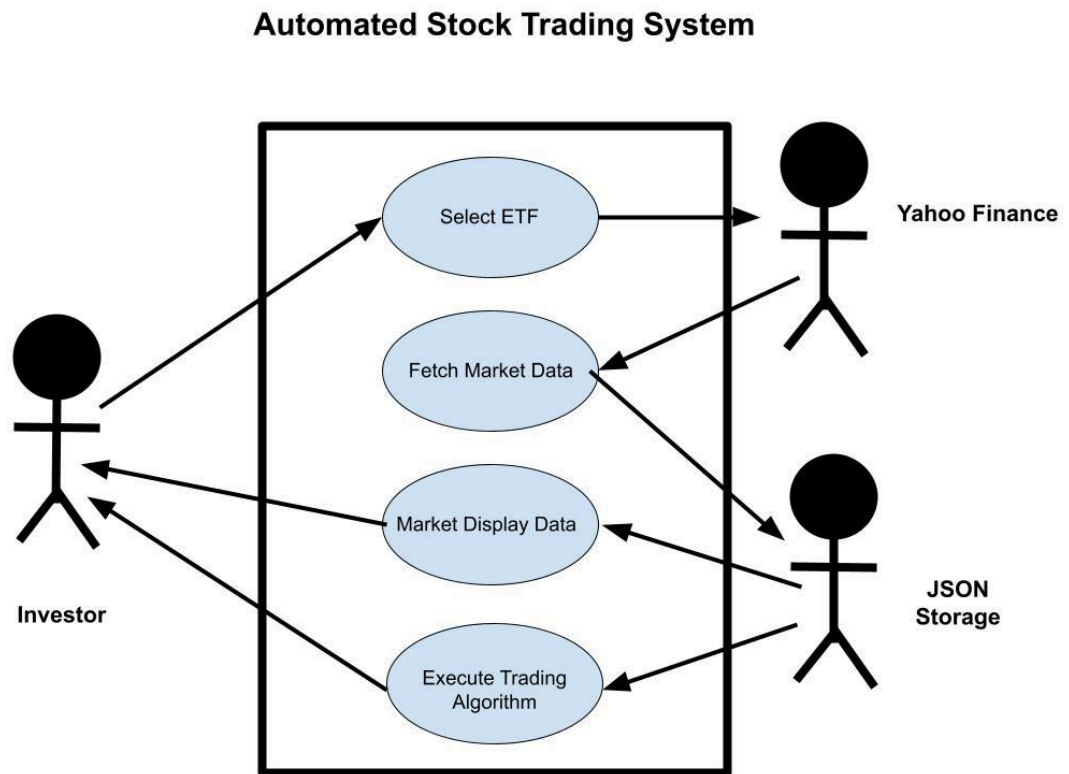
ID:	UC-3
Name:	Display Market Data
Description:	Show historical graph, print the latest market data, price % change from the day/month/year before, and historical price graph to the user
Primary Actor:	User
Secondary Actor:	System
Preconditions:	1) User has selected a valid symbol 2) Market data is available in a JSON format 3) User has specified a date range 4) Frontend components are loaded 5) Backend is awaiting a function call
PostConditions:	

	1) Historical graph is rendered 2) Price changes are calculated and displayed
Main Success Scenario: (Normal Flow)	1) User requests market data visualization 2) System retrieves data from JSON storage 3) System calculates required metrics: a) Daily price changes b) Monthly price changes c) Yearly price changes 4) System generates historical price graph 5) User is able to view the generated visualizations for their selected date range

ID:	UC-4
Name:	Execute Trading Algorithm
Description:	System runs trading algorithms based on the currently selected one in the frontend, generates trade signals, predicts stock based on each algorithm, logs trades (buy, sell, hold), and prints the data on the frontend for the user to easily see. Additionally, the user will be given an option to download the trade logs which contain all the data that is seen on the frontend for easy storage.
Primary Actor:	System
Secondary Actor:	User
Preconditions:	1) User has selected a valid symbol (FNGU or FNGD) 2) Historical data (.JSON) is up to date and available 3) Trading algorithms are configured and operational 4) System has sufficient resources for computation
PostConditions:	1) Trading signals are generated 2) Performance metrics are calculated 3) Performance metrics and stock predictions are saved in a .csv file

	4) Results from the .csv file are displayed on frontend 5) Trade log is available for download
Main Success Scenario: (Normal Flow)	1) User clicks on desired trading strategy button (SMA, BB, or MACD) 2) System acknowledges strategy selection 3) Trading System loads historical data for the selected ETF from JSON storage 4) Trading System applies selected algorithm to data 5) Trading System generates: a) Buy/sell signals b) Trade timing 6) System stores all trade logs during the algorithm's execution a) Buy/sell transactions 7) System calculates performance metrics: a) Total % returns b) Yield c) Initial vs final balance 8) System saves all the data to a .csv file 9) System displays results on frontend: a) Trade history b) Performance summary c) Historical signals 10) User reviews trading results 11) User clicks the download button 12) User downloads the .csv file

(c) a use case diagram



(d) product backlog

Use Case ID	Priority	Name of use case	Story Points	Value Points
UC-2	Very High	Fetch Market Data	6	1
UC-4	High	Execute Trading Algorithms	34	8
UC-3	Medium	Display Market Data	13	5
UC-1	Low	Select ETF	4	3

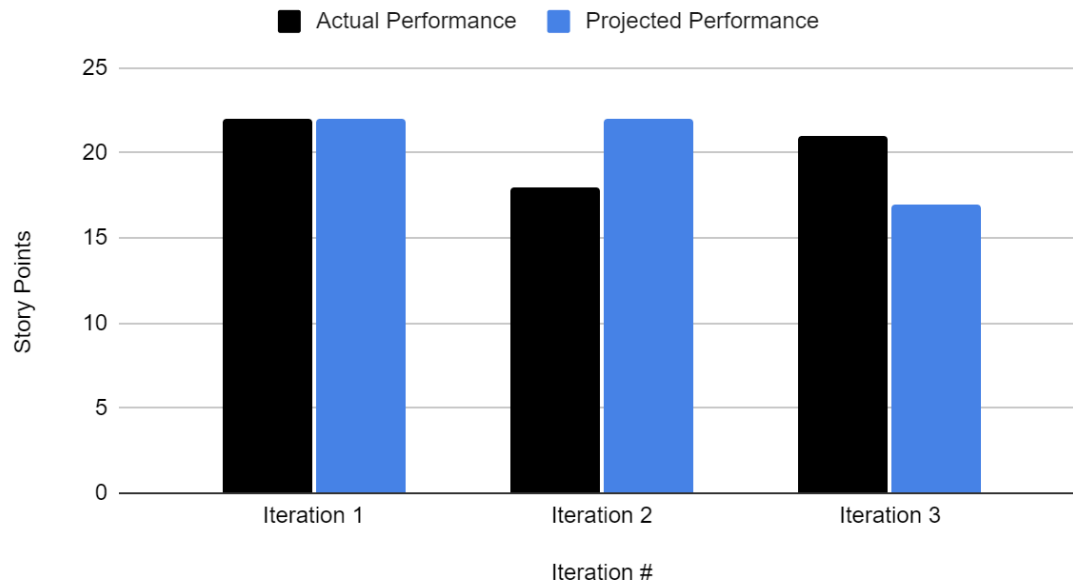
(e) release plan

Release Plan

	Iteration 0 October 8, 2024	Iteration 1 October 23, 2024	Iteration 2 November 4, 2024	Iteration 3 November 24, 2024	Iteration 4 December 9, 2024
	Task: Define project scope and goals	UC-1 Select ETF 1 story point	UC-1 Select ETF 3 story points	UC-3 Display Market Data 4 Story points	Task: Refactor the program to match criteria as well as for allow for easier maintainability and usability
	Task: Establish technical requirements				
	Task: Select trading strategies to automate	UC-2 Fetch Market Data 5 story points	UC-1 Fetch Market Data 1 Story point	UC-4 Execute Trading Algorithm 13 story points	Task: Simulation of live price after most recent market close
		UC-3 Display Market Data 3 story points	UC-3 Display Market Data 6 Story points		
		UC-4 Execute Trading Algorithm 13 story points	UC-4 Execute Trading Algorithm 8 story points		
Plan		22	22	17	
Capacity		22	18	21	

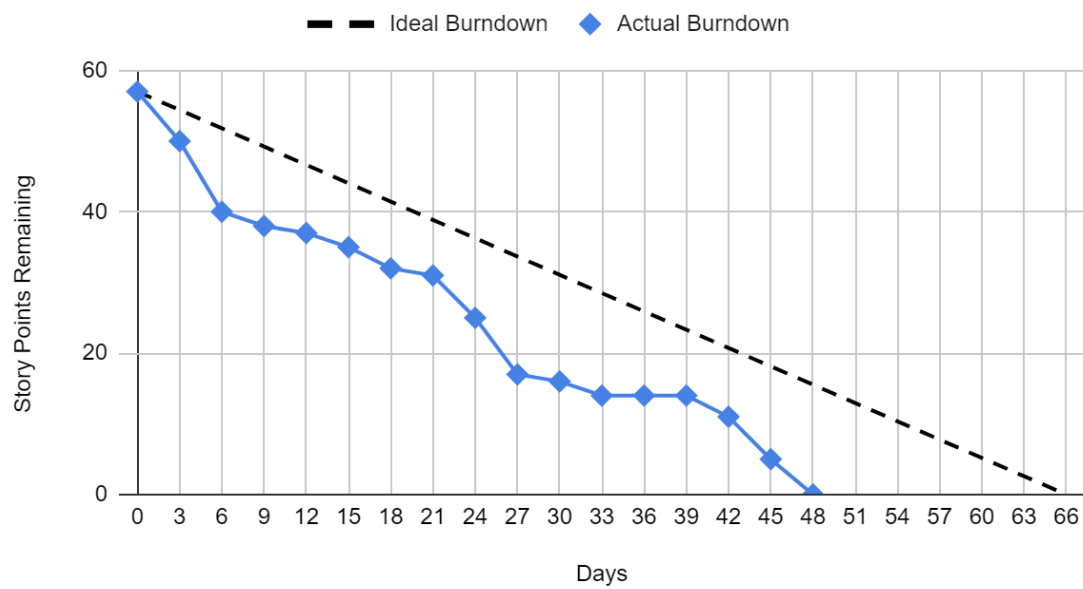
(f) velocity chart

Velocity Chart



(g) burndown chart

Sprint Burn-Down Chart



(h) CRC model

Frontend

App		
Attributes	Methods	Collaborators
selectedSymbol: Tracks the selected stock symbol month: Current month selection day: Current day selection year: Current year selection marketData: Contains fetched market data TRADING_ALGORITHMS: Stores the trading algorithm selected by the user tradeLog: Array of executed trades and their details finalBalance: Final account balance after trading totalGainLoss: Total profit/loss from all trades annualReturn: Percentage return on annual basis totalReturn: Total percentage return on investment	handleInputChange(event, setter, maxLength): Validates and updates date input fields validateDate(): Ensures dates are within valid ranges and formats downloadTradeLog(): Converts trade history to CSV and triggers download fetchData(): Retrieves market data from backend API runBacktest(algorithm): Executes trading strategy backtest with selected algorithm setSelectedSymbol(): sets the symbol selected that the user would like to see	StockChart TradeLogTable Server LivePrice

StockChart		
Attributes	Methods	Collaborators
chartData: Contains configured chart settings, data, and plugins dataPoints: Array of price data formatted to make the chart (date and closing price) buySignals: Array of buy signals for visualization sellSignals: Array of sell signals for visualization Style: marketDataChartStyle: Main chart styling chartTitleStyle: Title text styling chartContainerStyle: Main chart container styling noDataTextStyle: Error message styling	getCSSVariable(name): Retrieves CSS variables from Colors.css for the colors setChartData(): Configures and updates chart with new data and settings fixDrawingOrderPlugin(): Controls the drawing order of chart elements Chart Rendering: Dynamically displays the chart or an error message based	App

TradeLog		
Attributes	Methods	Collaborators
trades: Array of trade records containing date, price, shares, transaction amount, gain/loss, balance finalBalance: Final account balance totalGainLoss: Total profit/loss from all trades annualReturn: Percentage return on annual basis	Trade Table Rendering Logic: Displays trade history Performance Summary Rendering Logic: Shows summary of trading performance Trade Data Validation Logic: Makes sure trade data exists	App StockChart

totalReturn: Total percentage return on investment Style: tradelogHolder: Container styling tradelogText: Header text styling performanceSummaryGrid: Layout of performance metrics performanceItem: Individual metric styling performanceValue: Metric value styling gainLossColor: Color based on profit/loss		
---	--	--

LivePrice		
Attributes	Methods	Collaborators
symbol: The stock symbol for which the live price is displayed price: The latest fetched price for the given symbol isMarketHours: Indicates whether the current time falls within market hours Style: priceStyle: Styling for price display	getLastClosingPrice(): Fetches the last closing price of the given stock symbol from the server. useEffect(): Resets the price when the symbol changes. Fetches the initial price. Sets up a polling interval to get updates prices. Cleans up the interval when component unmounts or the symbol changes.	App Server

Backend

Server (API Handler)		
Attributes	Methods	Collaborators
app: Flask application instance routes: Defined endpoints /fetch_market_data /run_backtest /get_last_closing_price start_date: Start date(right now fixed at 2021-01-01)	fetch_market_data(): Handles POST requests for market data, validates symbol and dates, returns formatted market data or an error run_backtest(): Executes trading algorithm and returns the results, validates symbol, date and algorithm inputs, returns the backtest results (trade log and performance) get_last_closing_price(): Handles the POST request to get the last closing price of a stock symbol. Validates the symbol and fetches the price using price_simulator. Subscribes to price via price_subscriber price_update_callback(symbol, price): Callback for handling real time changes in the price	market_data tradingAlgos App price_subscriber

market_data_adapter		
Attributes	Methods	Collaborators
symbol: The symbol for which we are fetching the data start_date: Start date for historical data end_date: End date for historical data	fetch_data(): Fetches historical stock data using yfinance, filters the data, Formats the dates to ISO, returns filtered DataFrame save_data(): Saves market data using the decorated DataAccessService load_data(): Loads the previously saved market data	data_access_service trading_strategy yahoo_finance_adaptee validation_decorator

	live_price(): fetches the live price	
--	---	--

tradingAlgos		
Attributes	Methods	Collaborators
data: Pandas DataFrame containing the market data symbol: Stock symbol that we are trading algorithm: Selected trading strategy (SMA, BB, or MACD) balance: Current account balance (starts at initial balance) shares: Current number of shares held trade_log: List of executed trades and their details total_gain_loss: Total profit/loss from all trades annual_return: Percentage annual return total_return: Total percentage return on investment INITIAL_BALANCE: Starting balance (start at 100000) SMA Attributes: short_window: Short-term moving average window (default: 50). long_window: Long-term moving average window (default: 200).	calculate_signals(): Unified method to calculate trading signals (buy/sell) based on the selected algorithm: SMA: Buy when SMA50 crosses above SMA200, sell when SMA50 crosses below SMA200 MACD: Buy when MACD crosses above the Singal_Line, sell when MACD crosses below Singed_Line BB: Buy when price crosses below Lower_BB, sell when price crosses above Upper_BB run_backtest(): Runs the backtest for the selected algorithm handles buy/sell logic based on signals, tracks transactions and performance metrics updates the trade_log save_trades_to_csv(): Exports trade history and summary to a CSV file	market_data_adapter

SMA50: Short-term moving average values. SMA200: Long-term moving average values. MACD Attributes: short_ema: Short-term EMA period (default: 12). long_ema: Long-term EMA period (default: 26). signal_line: Signal line EMA period (default: 9). MACD: MACD line values. Signal_Line: Signal line values. BB Attributes: window: Period for calculating moving average (default: 20). num_std_dev: Number of standard deviations for bands (default: 2). Upper_BB: Upper Bollinger Band values. Lower_BB: Lower Bollinger Band values.		
--	--	--

price_publisher		
Attributes	Methods	Collaborators
_instance: Singleton instance of the PricePublisher class _subscribers: Dictionary mapping stock symbols to a list of subscriber callback function	subscribe(symbol: str, callback: Callable[[str, float], None]): Adds a callback function to the subscriber list for a specific stock symbol. If the symbol doesn't have an existing symbol then it will initialize an empty list. unsubscribe(symbol: str, callback: Callable[[str, float], None]): Removes a callback function from	simulate_market_price

	the subscriber list for a specific stock symbol. Makes sure that no actions can be taken if the callback or the symbol are not found notify(symbol: str, price: float): Invokes all subscribed callback functions for the specified stock symbol with the provided price. Also makes sure that only the relevant subscribers are notified.	
--	--	--

price_subscriber		
Attributes	Methods	Collaborators
symbol: The stock symbol to which this subscriber is listening for price updates publisher: Instance of PricePublisher used to manage the subscriptions and notifications	subscribe(callback: Callable[[str, float], None]): Subscribes to price updates for the symbol and registers the provided callback function with the PricePublisher unsubscribe(callback: Callable[[str, float], None]): Unsubscribes from the price updates for the symbol and removes the provided callback function from the PricePublisher	price_publisher LivePrice

simulate_market_price		
Attributes	Methods	Collaborators
_last_prices: A dictionary to store the most recent simulated price for both trading symbols _base_prices: A dictionary to store the initial closing price the MarketDataAdapter for both symbols	get_last_closing_price(symbol: str): Fetches the last closing price for the given symbol. If there is no price then it retrieves it using the MarketDataAdapter. If there is a base price then it will call simulated_price() to generate a new price. simulate_price(symbol: str):	market_data_adapter

	Simulates a price for a given symbol based on the price in <code>_last_prices</code> (applies a random % change of ± 0.01) and updates <code>_last_prices</code>.	
--	---	--

yahoo_finance_adaptee		
Attributes	Methods	Collaborators
	fetch_market_data(symbol: str, start_date: str, end_date: str): Fetches historical market data for the given symbol using the Yahoo Finance API. It get the Open, High, Low, Close, as well as Volume data and formats it. save_data(data: pd.DataFrame, filename: str): Saves the provided data to a JSON file and converts the DataFrame to a dictionary and writes it to file. load_data(filename: str): Loads market data from a JSON file and returns it as a DataFrame. get_live_price(symbol: str): Fetches the most recent closing price for the given symbol using the Yahoo Finance API (uses a 1 day period to retrieve the latest data)	market_data_adapter

validation_decorator		
Attributes	Methods	Collaborators
	fetch_market_data(symbol: str, start_date: str, end_date: str): Validates the input and fetches market data using decorated component. _validate_inputs(symbol: str, start_date: str, end_date: str): Checks if the symbol is valid, makes sure htat the start_date and end_date are in the correct format, makes sure the	market_data_adapter market_data_decorator

	end_date is after the start_date. _validate_data(data: pd.DataFrame): Makes sure that the input data contains all required columns (Date, Open, High, Low, Close, Volume) raises an error if any are missing.	
--	--	--

Testing

ServerIntegrationTest		
Attributes	Methods	Collaborators
app: Flask test client instance test_payloads: Test data for API calls Market data payload Invalid date payload	setUp(): Initializes test client and configures test environment test_fetch_market_data_endpoint(): Tests successful market data retrieval and verifies response structure and status code test_invalid_date_format(): Tests error handling for invalid date input, verifies error response and status code, validates error message content	Server unittest

TradingAlgorithmTest		
Attributes	Methods	Collaborators
test_data: Pandas DataFrame with sample market data contains Date, Open, High, Low, Close, Volume columns	setUp(): Initializes test data DataFrame and sets up test environment test_sma_backtest(): Tests SMA strategy execution and validates return value types (final_balance, trade_log, etc.) Makes sure the final balance is	trading_strategy unittest

	non-negative. Checks trade log structure and required fields (date, symbol, action, price, shares, transaction_amount, gain/loss, balance)	
--	--	--

(i) Architecture Diagram

